

Lecture 19: 11th march 2026:

Attention Mechanism and Transformers:

$X \xrightarrow{\phi} \mathbb{R}$: Representations / Embeddings.

* In a Neural Network

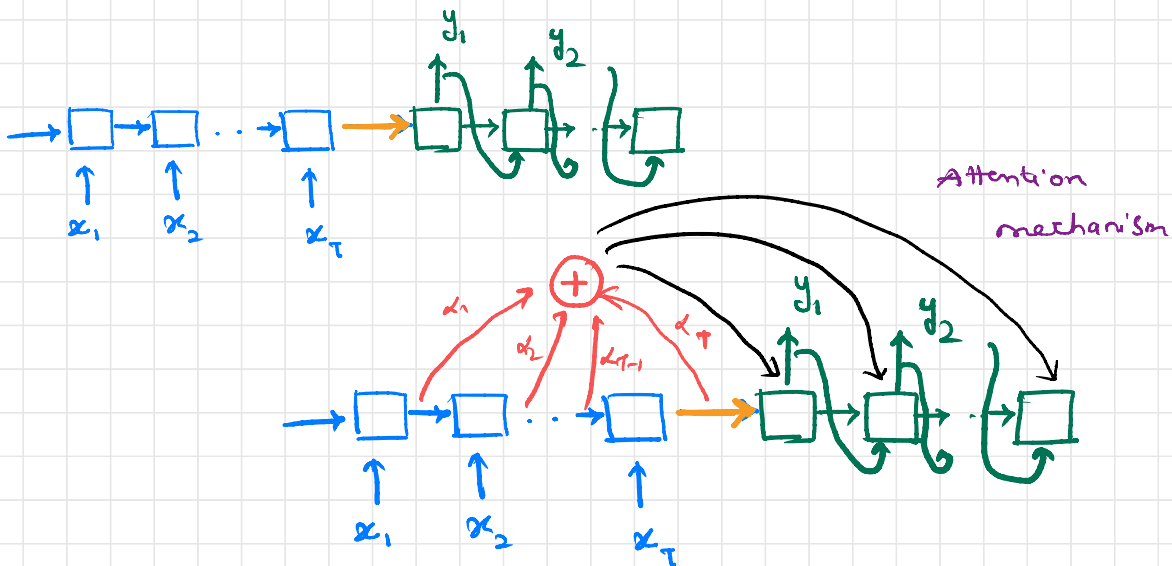
$$h(x) = \underbrace{\sigma(w_2 \sigma(w_1 x))}_{\phi(x)}$$

non linear Projection. / Learnable embedding

non linear projection with learnable Basis.

Attention Mechanism:

* Seq to Seq



$$y_t = F \left(y_{t-1}, \sum_{t=1}^T \alpha_t h_t \right)$$

Given a single data point

$$x = \{x_1, x_2, \dots, x_T\} \quad x_i \in \mathbb{R}^d \text{ (token)}$$

Objective is to learn projection/representations that encodes the interactions b/w the tokens.

* Let x be a matrix, having the data

$$x \in \mathbb{R}^{T \times d}$$

$$x = \begin{bmatrix} - & x_1 & - \\ - & x_2 & - \\ & \vdots & \\ - & x_T & - \end{bmatrix}_{T \times d}$$

Define Linear projection of x , using three matrices.

$$q = x w_q$$

$$w_q \in \mathbb{R}^{d \times d_q}$$

$$k = x w_k$$

$$w_k \in \mathbb{R}^{d \times d_k}$$

$$v = x w_v$$

$$w_v \in \mathbb{R}^{d \times d_v}$$

For single understanding

$$\text{Assume } d_k = d_v = d_k$$

Defining the Attention Vector:

Given a row of q , denoted by $Q \in \mathbb{R}^{d_q}$

Compute the following to get the attention.

$$i) \quad s_i = Q \cdot k_i^T \quad \forall \quad k_i \in K \quad i = 1, \dots, T.$$

Let $S = Q \cdot K^T$ a T -length vector

quantifying the similarity b/w
 Q & all rows of K .

$$\hat{s} = \frac{S}{\sqrt{d_k}} \\ \in \mathbb{R}^T$$

S : quantifying the similarity
b/w one token with all
other tokens.

$$\alpha = \text{softmax}(\hat{s}) \in \mathbb{R}^T$$

α : scaled similarities b/w Q & all
the elements of K .

Define Attention vector A_Q for Q as

$$A_Q = \sum_{i=1}^T \alpha_i v_i$$

$$\text{Attention}(q, k, v) = \left[\text{softmax} \left(\frac{q k^T}{\sqrt{d_q}} \right) \right] v$$

Attention.

$$X_{T \times d} \longrightarrow Z_{T \times d_g}$$

Multihead Attention:

* Learn multiple attention projections on x

Call them $z_1, z_2, \dots, z_m$

where $z_j = \text{Attention}(x)$

Define MHA

$$\text{MHA}(x) = \underbrace{\begin{bmatrix} z_1 & \dots & z_m \end{bmatrix}}_{T \times (m \times d_g)} \underbrace{w_z}_{(m \times d_g) \times d_g} \in \mathbb{R}^{T \times d_g}$$

$$h_\theta(x) : x \rightarrow y$$

regularize (θ)

$$h_\theta(x) = w_2 \sigma(w_1 x)$$

$$= w_2 z$$

$$\theta = \{w_1, w_2\}$$

$$z = \sigma(w_1 x)$$

$h_\theta(x)$ can be viewed as a func of θ & z .

as we have regularized θ , so we can regularize z .

Normalization are used to regularize z .

1) Batch normalization.

2) Layer normalization.

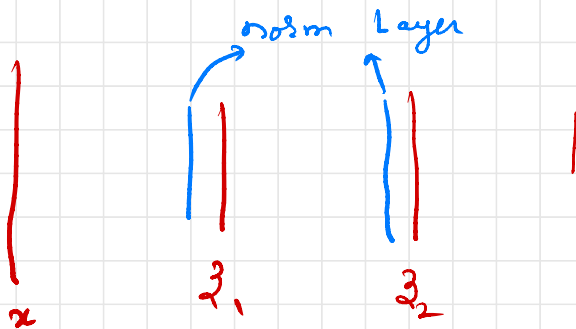
$$\hat{z} \leftarrow \left(\frac{z - \mu_2}{\sigma_2} \right) \gamma + \beta$$

$$\mu_2 = \frac{1}{B} \sum z_i$$

$$\sigma_2 = \text{Var}(z_i)$$

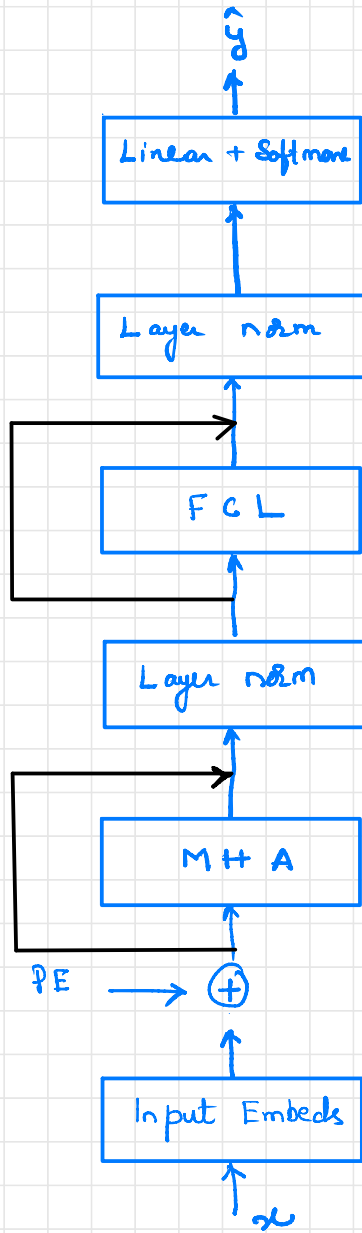
γ, β are learnt

To avoid co-variate shift we use normalization.



- * In batch norm μ & σ is computed by considering the example of a batch.
- * In layer norm μ & σ is computed across the dimension of data.
- * When $\beta = \mu$ & $\gamma = \sigma$, normalization can be undone.

Transformer Architecture:



multiple of
these are
stacked
one top of
another

Lecture 20: 16th Mar 2026:

Positional Embeddings:

consider a sequence $x = \{x_1, x_2, \dots, x_T\}$

temporal ordering is present in RNNs.

But not in Transformers.

* To incorporate the ordinality of the input sequence, another fixed "temporal variable" needs to be added.

* $t = \{0, 1, 2, \dots, T\}$

$$\hat{x}_i = x_i + t_i \quad \in \mathbb{R}^{d+1}$$

$$x_i \in \mathbb{R}^d$$

$$t_i \in \mathbb{R}$$

* time info may be ignored due to high dimension.

* Scalar 't' can be ignored and

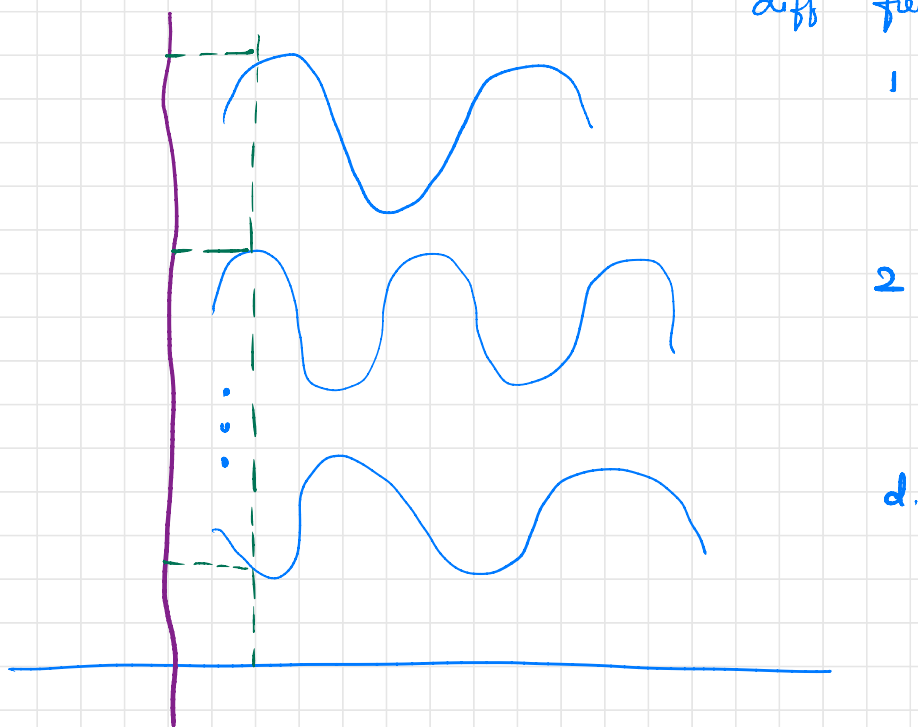
$\therefore$ fixed vector of dim d needs to be considered

$$f: t \rightarrow \hat{t} \quad \mathbb{R} \rightarrow \mathbb{R}^d$$

positional embeddings.

Eg: Sinusoidal Embeddings:

multiple Sins with
diff freq



$$\hat{t}(i, 2t) = \sin\left(\frac{i}{N^{2t/d}}\right)$$

$i = 0, \dots, d$

$$\hat{t}(i, 2t+1) = \cos\left(\frac{i}{N^{2t/d}}\right)$$

now we have

$$\hat{x}_i = x_i + \hat{t}_i$$

Transfer learning:

Given $x \in \mathbb{R}^d$, $y \in \mathbb{R}$

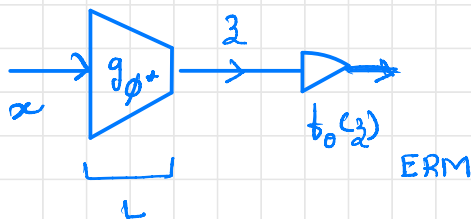
Suppose $g_{\phi^*}: X \rightarrow Y$

$g_{\phi^*}(\cdot)$ is a NN Learned via ERM
(pretrained NN)

* Since $g_{\phi^*}(\cdot)$ is a Composite func, output can be tapped at any layer.

* $z = g_{\phi^*}(x) \Big|_{\text{Layer-L}}$: Embedding of x
from g_{ϕ^*}

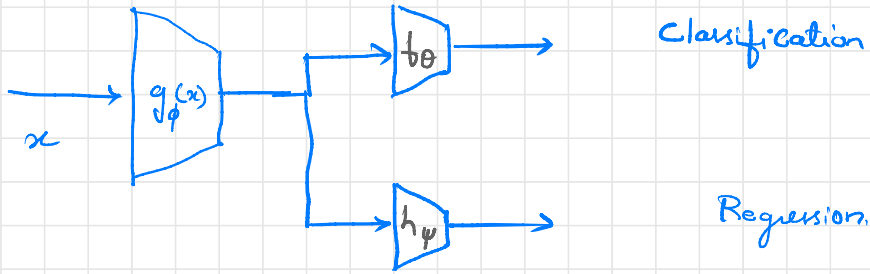
* Can use "z" for any further down-stream task



* pretrained initiation

* @ Only train $f_{\theta}(z)$

* Multi task Learning.



Knowledge Distillation:

Suppose $x, y \sim P_{xy}$

$g_\phi(\cdot)$ is trained on a task

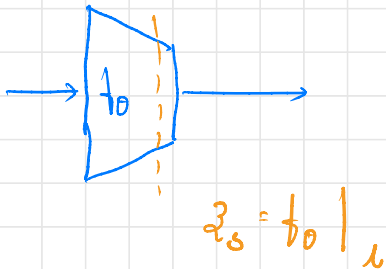
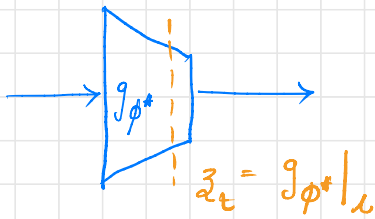
Let $f_\theta(\cdot)$ be another NN (typically with lesser parameters than g_ϕ)

g_ϕ : Teacher n/w

f_θ : Student n/w

Question: how to get the performance of Student n/w to match that of the teacher.

*



Ensure that $\dim(z_t) = \dim(z_s)$

Regularize the student η/ω s.t

$$D(\mathbf{p}_{z_t} \parallel \mathbf{p}_{z_s}) \text{ is minimized.}$$

* this will ensure that at layer L , feature learnt by f_0 @ layer L , is same as the one @ g_{ϕ} @ layer L

$$\hat{R}_s(\theta) = \argmin \left[D(\mathbf{p}_{z_t} \parallel \mathbf{p}_{z_s}) + \frac{1}{N} \sum \ell(\psi_0^L, y) \right]$$

Training Neural Network with Adaptive LR:

NNs are trained with ERM, via gradient descent.

Suppose $\hat{R}(\theta)$ is the ER, grad descent

$$\theta^{t+1} \leftarrow \theta^t - \alpha \nabla_{\theta} \hat{R}(\theta)$$

Stochastic Gradient Descent:

$$\tilde{R}(\theta) = \frac{1}{B} \sum_B \mathcal{L}(h_{\theta}(x), y)$$

where $B \subset D$, randomly sampled.

α is fixed across epochs & parameters.

Adaptive Learning Rates:

1) SGD with Momentum

$$\text{Define } m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$\text{where } g_t = \nabla_{\theta} \hat{R}(\theta)$$

$$\theta^{t+1} \leftarrow \theta_t - \alpha m_t$$

m_t : moving average of gradients.

Earlier time take larger step & later steps take

Smaller steps.

2) RMS Prop

look @ the accelerator which is g_t^2 , 2nd order gradient

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\theta^{t+1} \leftarrow \theta_t - \frac{\alpha}{\sqrt{v_t}} g_t$$

Scaling with inverse of g_t^2 ,

3) Adaptive momentum estimation: (Adam)

Combination of previous both.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\theta^{t+1} \leftarrow \theta_t - \frac{\alpha}{\sqrt{v_t}} m_t$$

β_1, β_2 : hyperparameter.